

Complessità di un algoritmo

L'esecuzione di un programma prevede un costo economico legato all'uso di risorse (memoria, rete etc) e di tempo di elaborazione

↳ complessità spaziale: utilizzo di risorse quali la memoria

↳ complessità temporale: tempo di esecuzione

↳ cresce al crescere della dimensione n dei dati in input

↳ legato al funzionamento di un algoritmo: numero di istruzioni eseguite.

Analisi asintotica

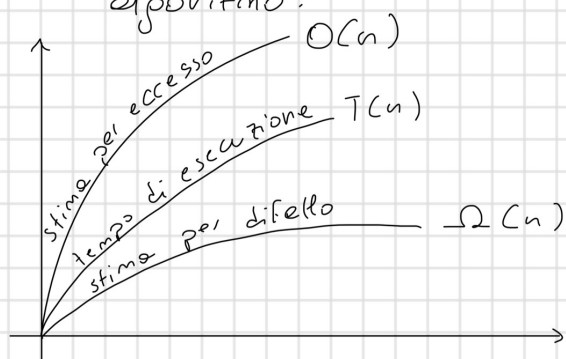
- In matematica si applica allo studio delle funzioni, per stabilire il tasso di crescita di una funzione confrontandola rispetto ad un'altra
- In informatica, il calcolo asintotico è utilizzato per analizzare la complessità di un algoritmo per stimare quanto aumenta al crescere delle dimensioni dell'input.

$T(n)$ → misura il numero di comandi eseguiti dall'algoritmo in funzione delle dimensioni n dell'input.

O : delimitazione superiore alla complessità dell'algoritmo

Ω : delimitazione inferiore alla complessità dell'algoritmo.

Θ : delimitazione sia superiore che inferiore alla complessità dell'algoritmo.



→ O $T(n) = O(f(n))$ → studio del caso peggiore

$$\exists c, n_0 > 0 \text{ t.c. } T(n) \leq c \cdot f(n) \quad \forall n \geq n_0$$

→ Ω $T(n) = \Omega(f(n))$ → studio del caso migliore

$$\exists c, n_0 > 0 \text{ t.c. } T(n) \geq c \cdot f(n) \quad \forall n \geq n_0$$

→ Θ $T(n) = \Theta(f(n))$ → studio del caso medio

$$\exists c_1, c_2, n_0 > 0 \text{ t.c. } c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n) \quad \forall n \geq n_0$$

Esempio di algoritmo di ricerca sequenziale

3	5	4	1	7	0
---	---	---	---	---	---

 · n x

```

for i = 1 to n
  if A[i] == x
    return x

```

CASO MIGLIORE: l'elemento è il primo dell'array

$$\text{complessità} = T(n) = \Omega(1)$$

CASO PEGGIORE: l'elemento non c'è nel vettore

$$\text{complessità} = T(n) = O(n)$$

CASO MEDIO: tutte le posizioni hanno uguale probabilità che l'elemento si trovi lì

$$\frac{1 + 2 + 3 + \dots + n}{n} = \sum_{i=1}^n \frac{1}{n} \cdot i = n + \frac{1}{2}$$

CLASSI DI COMPLESSITÀ

I $\left\{ \begin{array}{l} O(1) \rightarrow \text{COMPLESSITÀ COSTANTE: per gli algoritmi che eseguono sempre lo stesso numero di operazioni indipendentemente dalla dimensione dei dati} \\ O(kn) \quad k < 1 \rightarrow \text{COMPLESSITÀ SOTTOLINEARE: es. ricerca binaria o logaritmica} \end{array} \right.$

II $\left\{ \begin{array}{l} O(n) \rightarrow \text{COMPLESSITÀ LINEARE: per gli algoritmi che eseguono un numero di operazioni proporzionali ad n} \\ \text{es. ricerca sequenziale} \\ O(n \log n) \rightarrow \text{es. gli algoritmi di ordinamento ottimi} \end{array} \right.$

III $\left\{ \begin{array}{l} O(n^k) \quad k \geq 2 \rightarrow \text{es. Bubble sort } O(n^2) \\ O(k^n) \rightarrow \text{COMPLESSITÀ ESPONENZIALE: es. Torre di Hanoi } O(2^n) \end{array} \right.$

RICERCA BINARIA

→ Algoritmo di ricerca su un vettore ordinato di un elemento in input x .

→ Ad ogni step effettua il confronto dell'elemento da cercare x :

- ① l'elemento coincide con quello di mezzo; in tal caso l'algoritmo termina e viene tornata la posizione
- ② l'elemento è più piccolo di quello di mezzo; in tal caso si prosegue la ricerca sulla prima metà del vettore.
- ③ l'elemento è più grande di quello di mezzo; in tal caso si prosegue la ricerca sulla seconda metà del vettore.

Ricerca binaria ricorsiva

ricercaBinariaRic(V, x)

$n \leftarrow \text{length } V$

if $n == 0$ return -1 // elemento non trovato

$i \leftarrow \lceil n/2 \rceil$ // $\lceil \cdot \rceil$ superiore, dunque se l'array ha 9 elementi, quello di mezzo è il 5°

if $V[i] == x$ return i

else if $(x > V[i])$ return ricercaBinariaRic($V[i+1, n], x$)

else if $(x < V[i])$ return ricercaBinariaRic($V[1, i-1], x$)

Calcolo del costo: operazione costante, qui vediamo che è un'operazione che si fa ad ogni ripetizione

$$T(n) = \begin{cases} O(1) & \text{se } n=0 \\ O(1) + T(\lceil \frac{n-1}{2} \rceil) & \text{se } n \geq 1 \end{cases}$$

$\uparrow$ $\uparrow$
costo speso chiamata ricorsiva
all'interno
della funzione

Andiamo a calcolare utilizzando il **metodo d'iterazione**:

Si "srotola" la ricorsione, ottenendo una sommatoria dipendente dalla sola dimensione n .

$$T(n) = \begin{cases} O(1) & \text{se } n=0 \\ O(1) + T(\lceil (n-1)/2 \rceil) & \text{se } n \geq 1 \end{cases}$$

$$T(n) = \begin{cases} O(1) & \text{se } n=0 \\ O(1) + T(n/2) & \text{se } n \geq 1 \end{cases}$$

$$T(n) = O(1) + \underbrace{T(n/2)}_{O(1) + T(n/4)} = O(1) + O(1) + \underbrace{T(n/4)}_{O(1) + T(n/8)} = O(1) + O(1) + O(1) +$$

$$T(n/8) \dots$$

✓ ci fermeremo quando:

$$\frac{n}{2^i} = 1 \Rightarrow n = 2^i \Rightarrow i = \log_2 n$$

↓

spariscono perché corrisponderebbe a $C+C+C$, ma sappiamo che nella notazione O le costanti spariscono
→

$$= \log_2 n \cdot O(1) + T(1) = \log_2 n \cdot O(1) + O(1) + O(1) \approx O(\log_2 n)$$

Esempio 2

$$T(n) = \begin{cases} n + T(n/2) & n > 1 \\ 1 & \text{se } n = 1 \end{cases}$$

$$T(n) = n + T(n/2) = n + n/2 + T(n/4) = n + n/2 + n/4 + T(n/8) =$$

$$= \frac{n}{2^0} + \frac{n}{2^1} + \frac{n}{2^2} + T(n/2^3) = \sum_{i=0}^{k-1} \frac{n}{2^i} + T\left(\frac{n}{2^k}\right)$$

↑

ci fermiamo quando questo elemento è 1

$$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n$$

$$= \sum_{i=0}^{\log_2 n - 1} \frac{n}{2^i} + T(1) = n \sum_{i=0}^{\log_2 n - 1} \left(\frac{1}{2}\right)^i + T(1) = n \cdot \frac{(1 - (1/2)^{\log_2 n})}{1 - (1/2)} + 1 =$$

↓

dato che sappiamo che

$$\sum_{i=0}^n \alpha^i = \frac{1 - \alpha^{n+1}}{1 - \alpha}$$

$$= \frac{n \cdot (1 - (\frac{1}{2})^{\log_2 n})}{\frac{1}{2}} + 1 = 2n \left(1 - \frac{1}{2^{\log_2 n}}\right) + 1 = 2n \left(1 - \frac{1}{n}\right) + 1 =$$

$$= 2n \left(\frac{n-1}{n}\right) + 1 = 2n - 2 + 1 = 2n - 1 = O(n)$$

METODO DI SOSTITUZIONE - RICERCA LINEARE RICORSIVA

→ Dato un vettore V ed un elemento x in input, si vuole ricavare l'elemento x all'interno di V , fornendo la posizione.

→ Ricorsivamente si può scrivere la funzione di ricerca, considerando di volta in volta il vettore con lunghetta di un'unità più piccola

↳ Ad ogni chiamata ricorsiva si esegue il confronto con l'ultimo elemento del vettore

ricercaLineareRicorsiva(V , pos, x)

if pos < 0

return -1

if $V[pos] = x$

return pos

return ricercaLineareRicorsiva(V, pos - 1, x)

Calcoliamo la complessità col **METODO DELLA SOSTITUZIONE** ^{#elem del vettore}

Equazione di ricorrenza per algoritmo di ricerca lineare ricorsivo $\rightarrow T(n) = \begin{cases} O(1) & \text{se } n=1 \\ O(1) + T(n-1) & \text{se } n > 1 \end{cases}$

L'intuizione che sarà una complessità lineare, dunque $O(n)$ che equivale a dire dimostrare che $T(n) \leq kn$ dove k è una costante

$\rightarrow$ Il metodo di sostituzione ha come idea quella di "intuire" la soluzione di una relazione di ricorrenza ed usare l'induzione matematica per dimostrare che la soluzione è effettivamente quella intuita.

Riscriviamo l'equazione senza notazione asintotica:

$$T(n) = \begin{cases} d & \text{se } n=1 \\ c + T(n-1) & \text{se } n > 1 \end{cases}$$

Dobbiamo dimostrare che $T(n) = O(n)$

$$T(n) \leq kn \quad \forall n$$

Caso base: $n=1 \quad T(1) \leq k \Rightarrow d \leq k \quad \text{vero} \quad \forall k \geq d$

passo induttivo: per ipotesi induttiva sappiamo che $T(n-1) \leq k(n-1)$ e' VERO

$$T(n) \leq kn$$

$$T(n) = \underbrace{T(n-1)}_{\leq k(n-1)} + c \leq k(n-1) + c = kn - k + c =$$

$$= kn - (k - c) \leq kn$$

$$k - c \geq 0 \Leftrightarrow k \geq c$$

$$T(n) \leq kn$$

$$\forall k \geq c$$

Quindi $T(n) = O(n)$ si verifica quando $k \geq c$ e $k \geq d$

Esempio

$$T(n) = \begin{cases} n + T(n/2) & n > 1 \\ 1 & \text{se } n = 1 \end{cases}$$

$$T(n) = O(n) \Rightarrow T(n) \leq cn$$

$$\text{CASO BASE } n=1 \quad T(1) \leq c \Rightarrow 1 \leq c \quad \forall c \geq 1$$

$\downarrow$

$$T(n') \leq cn' \quad \text{IPOTESI INDUTTIVA}$$

$$\forall n' < n$$

$$T(n) \leq cn$$

$$T(n) = n + T(\underbrace{n/2}_{n/2 < n}) \leq n + c(n/2) =$$

$$= n(1 + \frac{c}{2}) \leq cn \Leftrightarrow 1 + \frac{c}{2} \leq c$$

$$\frac{2+c}{2} \leq c \Rightarrow 2 + \underbrace{c}_{\leq 2c} \leq 2c = c \geq 2 \quad \forall c \geq 2$$

ALBERI DI RICORSIONE

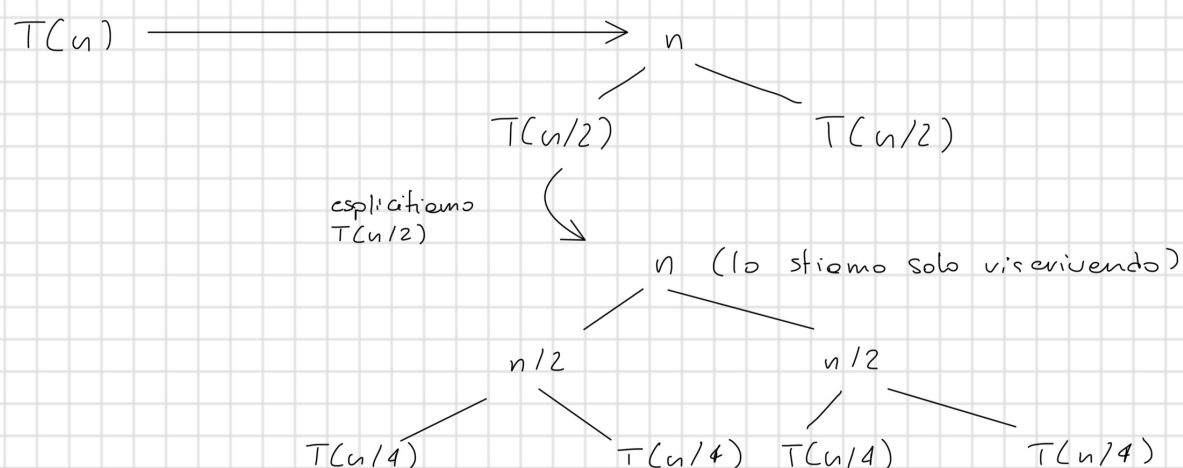
- Si tratta di un metodo alternativo al metodo iterativo
- Un albero di ricorsione è un albero in cui ogni nodo rappresenta il costo di un sottoproblema in base alla specifica diametro ricorsiva.

Punti di attenzione

- 1) Quanti nodi ci sono al livello i
- 2) Qual'è il costo di un nodo al livello i
- 3) Qual'è il costo complessivo dei nodi al livello i
- 4) Qual'è il numero di livelli
- 5) Qual'è il costo complessivo di $T(n)$

Alberi di ricorsione - esempio 1

$$T(n) = \begin{cases} n + 2T(n/2) & \text{se } n > 1 \\ 1 & \text{se } n = 1 \end{cases}$$

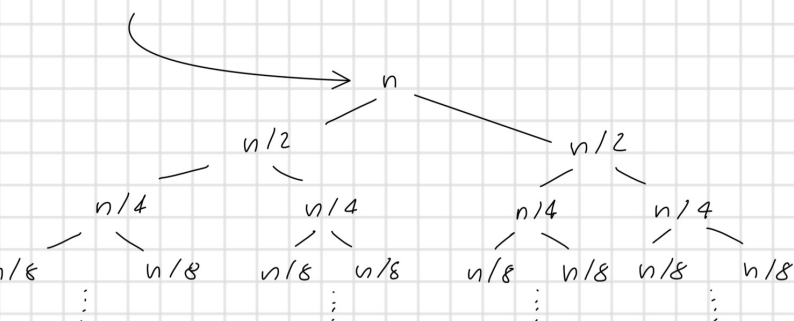


Livello 0

Livello 1

Livello 2

Livello 3



Livello k $T(1)$ $T(1)$ $\dots$ $T(1)$

Gi fermiamo a $T(1)$ perché l'equazione ci diceva che, nel caso in cui $n=1$, il costo è costante.

Rispondiamo alle domande che ci siamo posti:

1) Quanti nodi abbiamo in ogni livello?

$$0 \rightarrow 1 \rightarrow 2^0$$

$$1 \rightarrow 2 \rightarrow 2^1$$

$$2 \rightarrow 4 \rightarrow 2^2$$

$$3 \rightarrow 8 \rightarrow 2^3$$

$$\vdots$$

$$i \rightarrow 2^i$$

2) Qual'è il costo di ogni nodo?

Il costo $n/2^i$

3) Qual'è il costo complessivo al livello i ?

Ci basta fare il costo di un nodo al livello i per il n° di nodi al livello i .

$$\frac{n}{2^i} \cdot 2^i = n$$

4) Qual'è il costo complessivo del nostro algoritmo?

$$T(n) = (\# \text{livelli} + 1) \cdot n$$

perché siamo partiti da 0

$$= (k+1) \cdot n$$

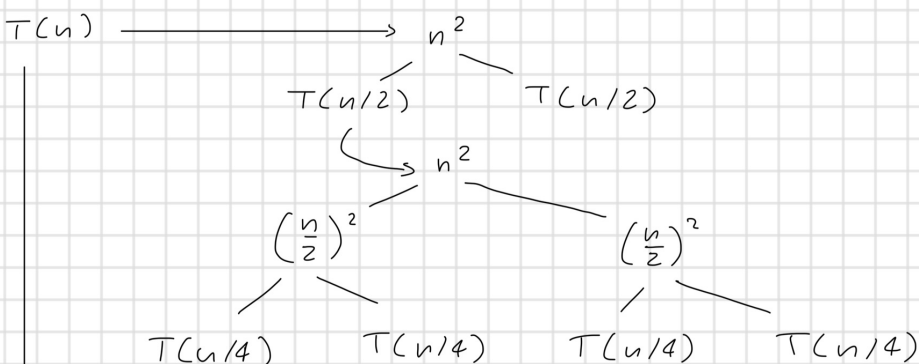
dobbiamo trovare k

$$\frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$$

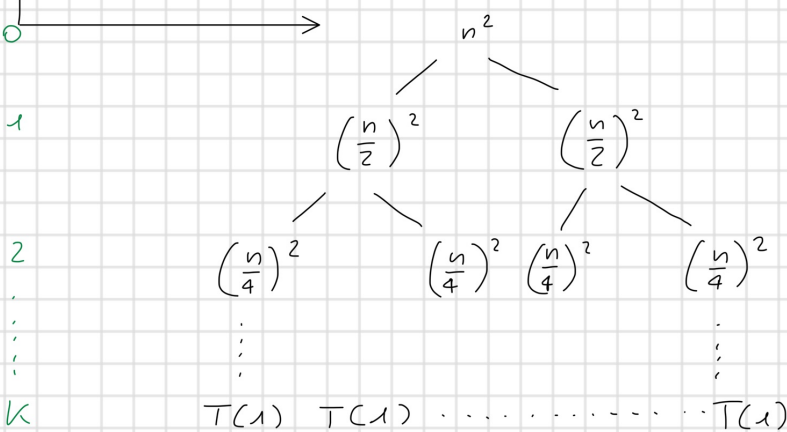
$$\text{Quindi } T(n) = (\log_2 n + 1) \cdot n = n \log n$$

Alberi di ricorsione - esempio 2

$$T(n) = \begin{cases} 2T(n/2) + n^2 & n > 1 \\ O(1) & n = 1 \end{cases}$$



avviveremo a questo albero



1) Quanti nodi abbiamo ad ogni livello?

$\rightarrow 1 \rightarrow 2^0$

$\rightarrow 2 \rightarrow 2^1$

$\rightarrow 4 \rightarrow 2^2$

$\vdots$

$\rightarrow k \rightarrow 2^k$

2) Qual'è il costo di ogni nodo?

$\left(\frac{n}{2^k}\right)^2 \rightarrow$ costo del singolo nodo

3) Qual'è il costo complessivo al livello i?

$2^k \cdot \left(\frac{n}{2^k}\right)^2 = 2^k \cdot \frac{n^2}{2^{2k}} = \frac{n^2}{2^k}$

4) Qual'è il costo complessivo del nostro algoritmo?

Come prima dobbiamo trovare k

$\left(\frac{n}{2^k}\right)^2 = 1 \Rightarrow \frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n$

$T(n) = \sum_{i=0}^{\log_2 n} \frac{n^2}{2^i} = n^2 \sum_{i=0}^{\log_2 n} \left(\frac{1}{2}\right)^i = \sum_{i=0}^{\log_2 n} \left(\frac{1}{2}\right)^i = \frac{1 - \left(\frac{1}{2}\right)^{\log_2 n + 1}}{1 - \frac{1}{2}} = \Theta(n^2)$

ricordando la
somma geometrica:

$\sum_{i=0}^N q^i = \frac{1 - q^{N+1}}{1 - q}$

dopo vari passaggi
troviamo che il valore
è circa 1 dunque

perché abbiamo:

- Ω (limite inferiore):
noi sappiamo che il costo
dell'albero che il costo
non sarà mai inferiore a
 n^2 . $\Omega(n^2)$
- O (limite superiore):
dal calcolo finale vediamo
che il costo non supererà
mai $2n^2$. $O(n^2)$
- Abbiamo dunque un muro,
tipico caso medio Θ

TEOREMA MASTER

Viene utilizzato come metodo per risolvere equazioni di ricorrenza di algoritmi ricorsivi basati sulla tecnica del **divide et impera**.

- **dividi**: per dividere il problema di dimensioni n , in a sottoproblemi di dimensione n/b ($b > 1$)

- **risolvi**: i sotto problemi ricorsivamente

- **ricombina**: le soluzioni

Sia $f(n)$ il costo della funzione escluse le chiamate ricorsive, inclusi i tempi per la divisione e la ricombinazione:

$$T(n) = \begin{cases} aT(n/b) + f(n) & n > 1 \\ \Theta(1) & n = 1 \end{cases}$$

Complessità del sottoproblema

Teorema Master - Soluzioni

$$T(n) = \begin{cases} aT(n/b) + f(n) & n > 1 \\ \Theta(1) & n = 1 \end{cases}$$

1) $T(n) = \Theta(n^{\log_b a})$ se $f(n) = O(n^{\log_b a - \epsilon})$ per qualche ϵ

2) $T(n) = \Theta(n^{\log_b a} \log n)$ se $f(n) = \Theta(n^{\log_b a})$

3) $T(n) = \Theta(f(n))$ se $f(n) = \Omega(n^{\log_b a + \epsilon})$ per qualche ϵ

$f(n)$ soddisfa la **condizione di regolarità**

$$af(n/b) \leq cf(n) \quad (c > 1)$$

Esempio 1

$$T(n) = T(n/2) + \Theta(1)$$

| riscrivo l'equazione in notazione generica

$$= aT(n/b) + f(n)$$

$$a = 1 \quad b = 2 \quad f(n) = \Theta(1)$$

$$f(n) = \Theta(n^{\log_b a}) \rightarrow \text{ricadiamo nel secondo caso}$$

$$= \Theta(n^{\log_2 1}) = \Theta(n^0) = \Theta(1)$$

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$$

Esempio 2

$$T(n) = 3T(n/3) + n$$

$$= aT(n/b) + f(n)$$

$$a = 3 \quad b = 3 \quad f(n) = n$$

$$f(n) = \Omega(n^{\log_b a + \epsilon}) = \Omega(n^{\log_3 3 + \epsilon}) = \Omega(n^{\frac{1}{2} + \epsilon}) = \Omega(n)$$

↓
dobbiamo scegliere
 $\epsilon = \frac{1}{2}$

Nel terzo caso va anche dimostrata la **condizione di regolarità**

$$af(n/b) \leq cf(n)$$

$$3f(n/3) \leq c \cdot n \iff 3 \cdot \frac{n}{3} \leq c \cdot n \quad \text{con } c = \frac{1}{3}, \text{ la disuguaglianza è soddisfatta}$$

Quindi siamo nel 3° caso:

$$T(n) = \Theta(f(n)) = \Theta(n)$$

Esempio 3

$$T(n) = 9T(n/3) + n$$

$$= aT(n/b) + f(n)$$

$$a = 9 \quad b = 3 \quad f(n) = n$$

$$f(n) = O(n^{\log_b a - \epsilon}) = O(n^{\log_3 9 - \epsilon}) =$$

$$= O(n^{2 - \epsilon}) \xrightarrow{\epsilon = 1} O(n^{2-1}) = O(n)$$

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n^2)$$

Limiti teorema master:

1. La forma deve essere perfetta:

Il teorema si applica solo ed esclusivamente alle ricorrenze che hanno l'esatta forma:

$$T(n) = aT(n/b) + f(n)$$

Se l'equazione non è riducibile a questa forma, il teorema è inutile. Ad esempio, non possiamo usarlo per equazioni con sottrazioni come $T(n) = T(n-1) + n$ (qui bisogna usare il metodo iterativo o la sostituzione).

2. I vincoli sulle costanti

Non possiamo dividere per zero e noi sappiamo che le costanti sono vincolate da questi valori:

- $a \geq 1$: L'algoritmo deve generare almeno un sottoproblema.
- $b > 1$: Dobbiamo effettivamente impicciolire il problema. Se $b = 1$, stiamo dividendo per 1, stiamo chiamando la ricorsione sulle stesse mole di dati.

3. Sottoproblemi sbilanciati

Il teorema Master dà per scontato che il problema venga diviso in blocchi tutti uguali. Ma se un algoritmo divide il problema in parti sbilanciate, ad esempio:

$$T(n) = T(n/3) + T(2n/3) + n$$

Il teorema Master fallisce. Qui l'unico modo è disegnare l'albero di ricorsione, calcolare i costi dei rami e fare la sommatoria.

4. I buchi neri tra i 3 casi

Questa è la trappola mortale della teoria. I 3 casi Master non coprono tutto l'universo matematico. Richiedono che $f(n)$ sia polinomialmente più piccolo di $n^{\log_b a}$. Se la differenza tra le due funzioni non è un polinomio ma, ad esempio, un logaritmo, l'equazione cade in una sorta di buco nero tra il 1° ed il 2°, oppure tra il 2° ed il 3°. In queste zone il teorema fallisce.